Ex3a: Model Planning and Building - Classification

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate basic machine learning models using real-world datasets.

Aim: To build Logistic Regression, Decision Tree and Random Forest models to predict customer churn and compare their performance.

Problem Statement:
A telecommunications company has noticed that a growing number of customers are discontinuing its services, resulting in reduced revenue and increased customer acquisition costs. After completing data preprocessing and exploratory analysis, the company now aims to develop predictive models that can identify customers who are likely to churn. Early prediction of customer churn will enable the company to implement targeted retention strategies, improve customer satisfaction, and reduce business losses.

As a Data Scientist, your task is to build and compare multiple classification models to predict whether a customer will churn. You will train different machine learning algorithms, evaluate their performance using appropriate classification metrics, and identify the most effective model for customer churn prediction. Finally, interpret the important factors influencing customer churn and communicate your findings through suitable visualizations and performance comparisons.

Tasks to Perform:

1. Use CustomerChurn_Preprocessed.csv file which you created in Ex2.
2. Split the cleaned dataset into Training (80%) and Testing (20%) datasets.

Model Building

3. Train a Logistic Regression classifier and predict the customer churn on the test dataset.
4. Train a Decision Tree Classifier and predict the customer churn on the test dataset.
5. Train a Random Forest Classifier and predict the customer churn on the test dataset.

Model Evaluation

6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, F1-Score.
7. Compare the performance of all three models using a single bar chart showing the evaluation metrics.
8. Identify the best-performing model based on the evaluation results.

Business Recommendation

9. Based on the best-performing model, provide three recommendations that the telecommunications company can implement to reduce customer churn.

Solution:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

df = pd.read_csv("CustomerChurn_Preprocessed.csv")
X, y = df.drop(columns="Churn"), df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
}
results = {}
for name, m in models.items():
    pred = m.fit(X_train, y_train).predict(X_test)
    print(name)
    print(confusion_matrix(y_test, pred))
    results[name] = {
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1-Score": f1_score(y_test, pred)}
res = pd.DataFrame(results).T.round(3)
print(res)

res.plot(kind="bar", figsize=(9, 5), rot=0)
plt.title("Model Comparison")
plt.ylabel("Score")
plt.show()
print("Best model (by F1):", res["F1-Score"].idxmax())

rf = models["Random Forest"]
print(pd.Series(rf.feature_importances_, X.columns)
        .sort_values(ascending=False).head(5).round(3))
```

Output:

Model	Accuracy	Precision	Recall	F1-Score	Confusion matrix
Logistic Regression	0.806	0.658	0.561	0.606	[[926, 109], [164, 210]]
Decision Tree	0.717	0.468	0.489	0.478	[[827, 208], [191, 183]]
Random Forest	0.783	0.610	0.503	0.551	[[915, 120], [186, 188]]

Logistic Regression performs best on all the metrics.

Recommendations:

1. Offer discounts to month-to-month customers to move them to one or two year contracts.
2. Give extra attention to new customers, since churn is higher in the early months.
3. Promote online security and tech support services, as customers without them churn more.

Result:

Logistic Regression gave the best result with an accuracy of 0.806 and an F1-score of 0.606, followed by Random Forest and Decision Tree.

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Ex3b: Model Planning and Building - Regression

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate regression models using a real-world dataset.

Aim: To build Linear, Polynomial and SVR models to predict sales from advertising spend and compare them using MAE, MSE, RMSE and R2 score.

Problem Statement:
A marketing agency wants to help a retail company optimize its advertising budget. The company invests in three advertising channels—TV, Radio, and Newspaper—and wants to understand how these advertising expenditures influence product sales. By developing predictive models, the company aims to estimate future sales based on advertising investments and identify the most suitable regression model for accurate sales prediction.

As a Data Scientist, your task is to build and compare multiple regression models using the Advertising dataset. Evaluate the performance of each model using appropriate regression metrics and communicate the findings through visualizations to support business decision-making.

Tasks to Perform:

1. Download the Advertising Dataset from Kaggle and load it into a Pandas DataFrame.
2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.
3. Select: TV, Radio, Newspaper as Input Features (X), Sales as Target Variable (y).

Perform Exploratory Data Analysis:

4. Generate a Pair Plot for the selected features.
5. Plot a Correlation Heatmap.
6. Identify the feature most strongly correlated with Sales.

Model Planning and Model Building

7. Split the dataset into Training (80%) and Testing (20%) dataset.
8. Build a Linear Regression model and predict Sales for the test dataset.
9. Build a Polynomial Regression model (Degree = 2) and predict Sales for the test dataset.
10. Build a Support Vector Regression (SVR) model with the RBF kernel and predict Sales for the test dataset.

Model Evaluation

11. Evaluate all three models using: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 Score
12. Compare the performance of all models using a single grouped bar chart.
13. Plot Actual vs Predicted Sales for the best-performing model.

Model Interpretation

14. Identify the model with the lowest prediction error.
15. Suggest **three recommendations** to help the company optimize its advertising budget based on the model results.

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.svm import SVR
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             r2_score)

df = pd.read_csv("advertising.csv")
print(df.head())
print(df.tail())
print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe())

X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]

# EDA
sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.show()
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()
print(df.corr()["Sales"].drop("Sales").sort_values(ascending=False))

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Models
lr = LinearRegression().fit(X_train, y_train)
pred_lr = lr.predict(X_test)

poly = PolynomialFeatures(degree=2)
pr = LinearRegression().fit(poly.fit_transform(X_train), y_train)
pred_pr = pr.predict(poly.transform(X_test))

sc = StandardScaler().fit(X_train)
svr = SVR(kernel="rbf").fit(sc.transform(X_train), y_train)
pred_svr = svr.predict(sc.transform(X_test))

# Evaluation
preds = {"Linear": pred_lr, "Polynomial": pred_pr, "SVR": pred_svr}
rows = {}
for n, p in preds.items():
    mse = mean_squared_error(y_test, p)
    rows[n] = {"MAE": mean_absolute_error(y_test, p), "MSE": mse,
               "RMSE": np.sqrt(mse), "R2": r2_score(y_test, p)}
res = pd.DataFrame(rows).T.round(3)
print(res)
res.plot(kind="bar", figsize=(9, 5), rot=0)
plt.title("Regression Model Comparison")
plt.show()

best = res["RMSE"].idxmin()
print("Lowest error model:", best)
```

```
plt.scatter(y_test, preds[best])
plt.plot([y.min(), y.max()], [y.min(), y.max()], "r--")
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title(f"Actual vs Predicted ({best})")
plt.show()
```

Output:

The dataset has 200 rows with no missing or duplicate values. TV has the highest correlation with Sales (0.78), followed by Radio (0.58) and Newspaper (0.23).

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	1.461	3.174	1.782	0.899
Polynomial Regression	0.526	0.413	0.643	0.987
SVR (RBF)	1.162	2.862	1.692	0.909

Polynomial Regression has the lowest error, so the Actual vs Predicted plot is drawn for it.

Recommendations:

1. Spend the most on TV advertising, since it has the strongest effect on sales.
2. Use Radio along with TV, as the combination gives better sales.
3. Reduce the Newspaper budget because its effect on sales is very small.

Result:

Polynomial Regression gave the lowest error (RMSE 0.643) and the highest R2 score (0.987) among the three models.